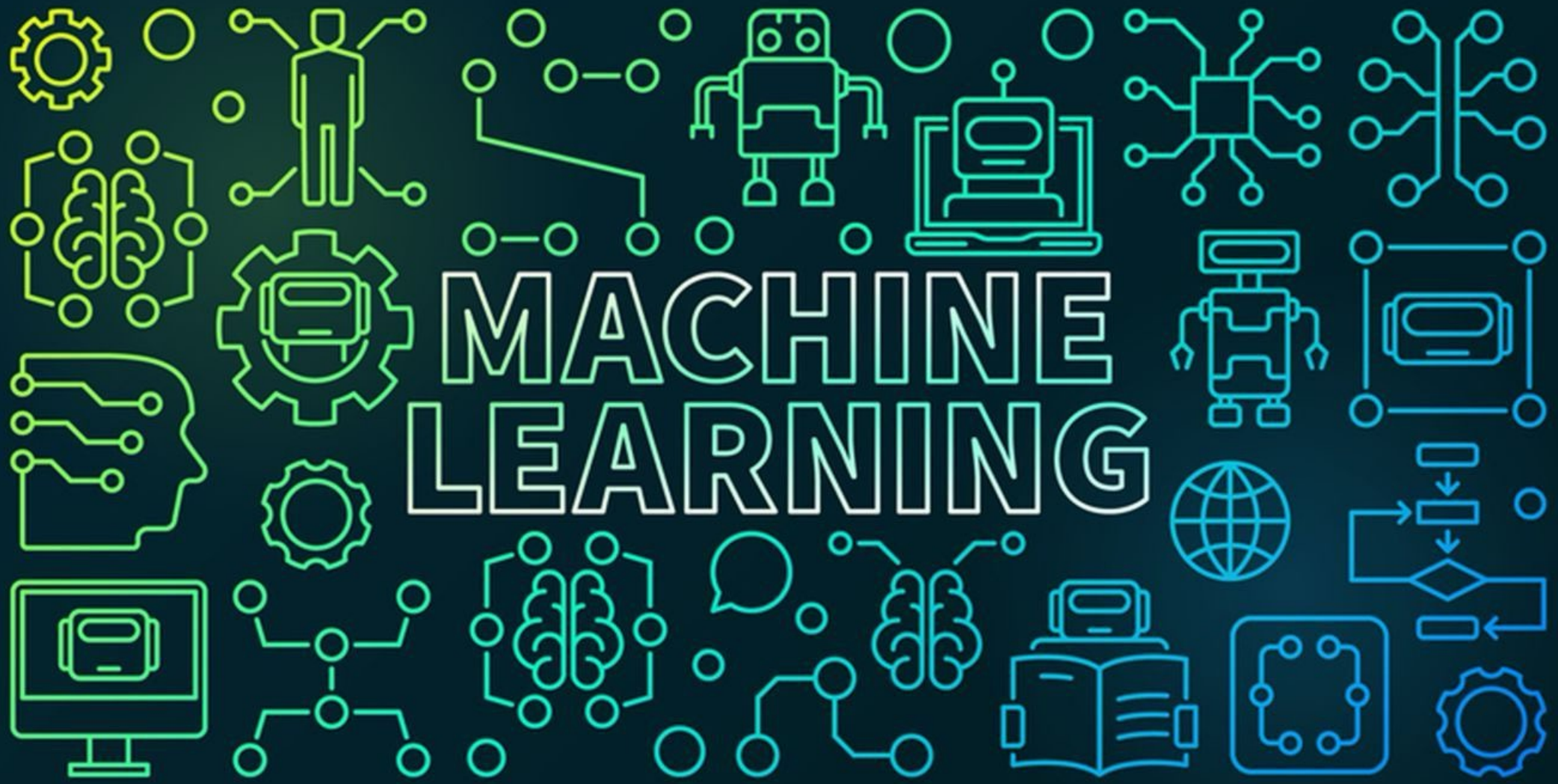



Muhammad Atif Saeed (Lecturer DS & AI)

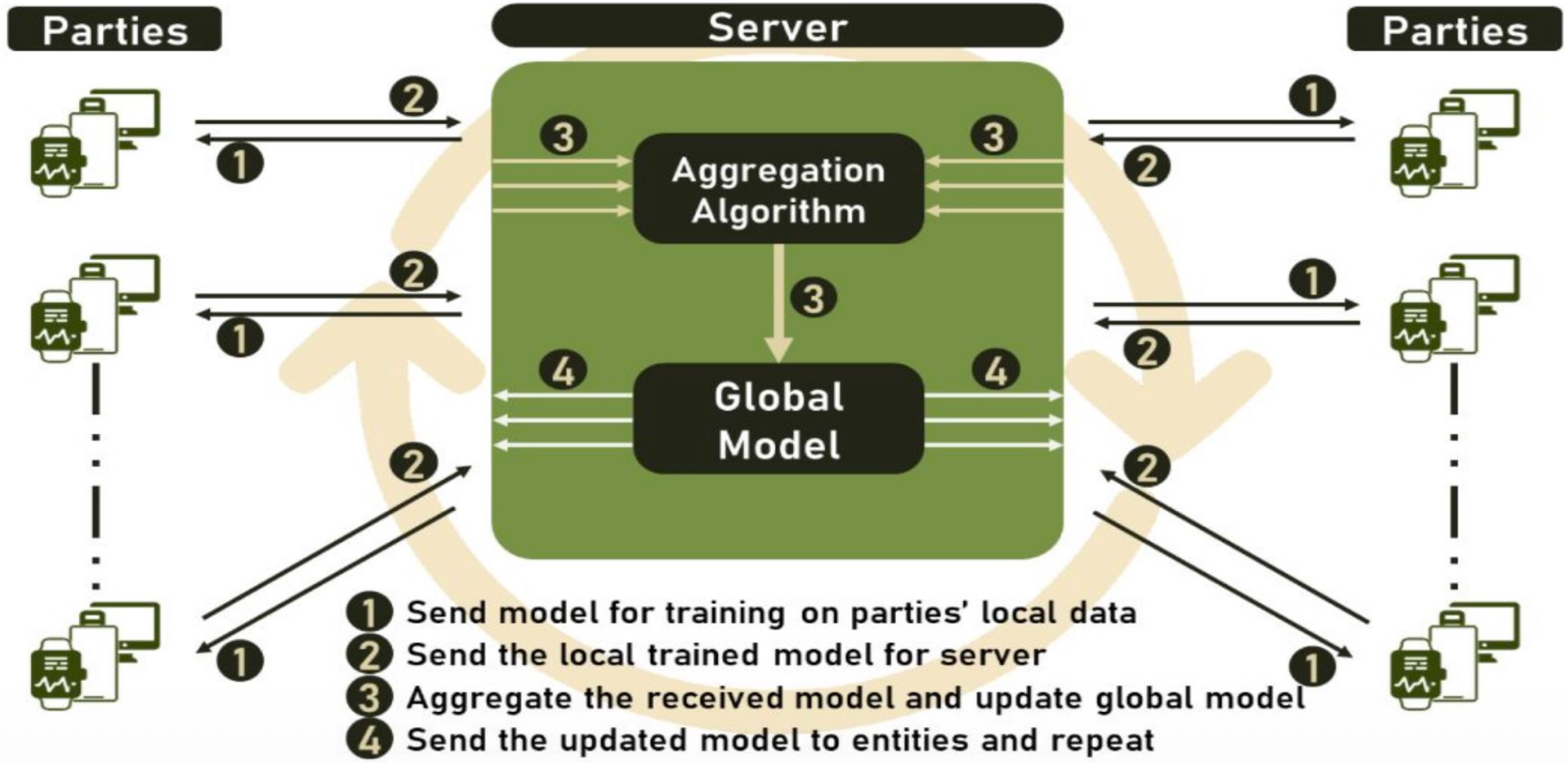
# Federated learning

- **Federated learning(FL)** is a framework used to train a shared global model on data across distributed devices while the data does not leave the device at any point.

# Underlying Architecture

- Central Server: the entity responsible for managing the connections between the entities in the FL environment and for aggregating the knowledge acquired by the FL clients;
- Parties (Clients): all computing devices with data that can be used for training the global model, including but not limited to: personal computers, servers, smartphones, smartwatches, computerized sensor devices, and many more;
- Communication Framework: consists of the tools and devices used to connect servers and parties and can vary between an internal network, an intranet, or even the Internet;
- Aggregation Algorithm: the entity responsible for aggregating the knowledge obtained by the parties after training with their local data and using the aggregated knowledge to update the global model.

# The learning process



# Exchanging Models, Parameters, or Gradients

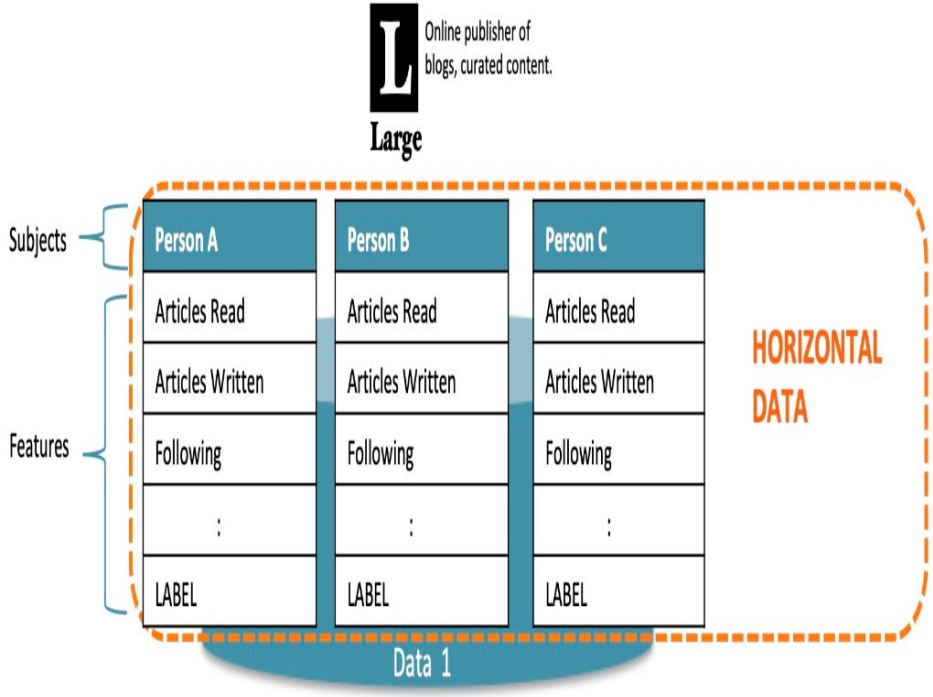
- Exchanging Models: This is the classical approach, where models are exchanged between server and clients.
- Exchanging Gradients: Instead of submitting the entire model to the server, clients in this method submit only the gradients they compute locally.
- Exchanging Model Parameters : This concept is mainly tied to neural networks where model parameters and weights are usually used interchangeably. Parameters, sometimes called weights.
- Hybrid Approaches: Two or more of the above methods can be combined to form a hybrid strategy that is particularly suited to a particular application or environment.

# The categorization of federated learning

- Federated learning enables collaborative model training without sharing raw data.
- It's categorized based on feature overlap in client datasets:
  - Horizontal Federated Learning (HFL).
  - Vertical Federated Learning (VFL).
  - Federated Transfer Learning (FTL).

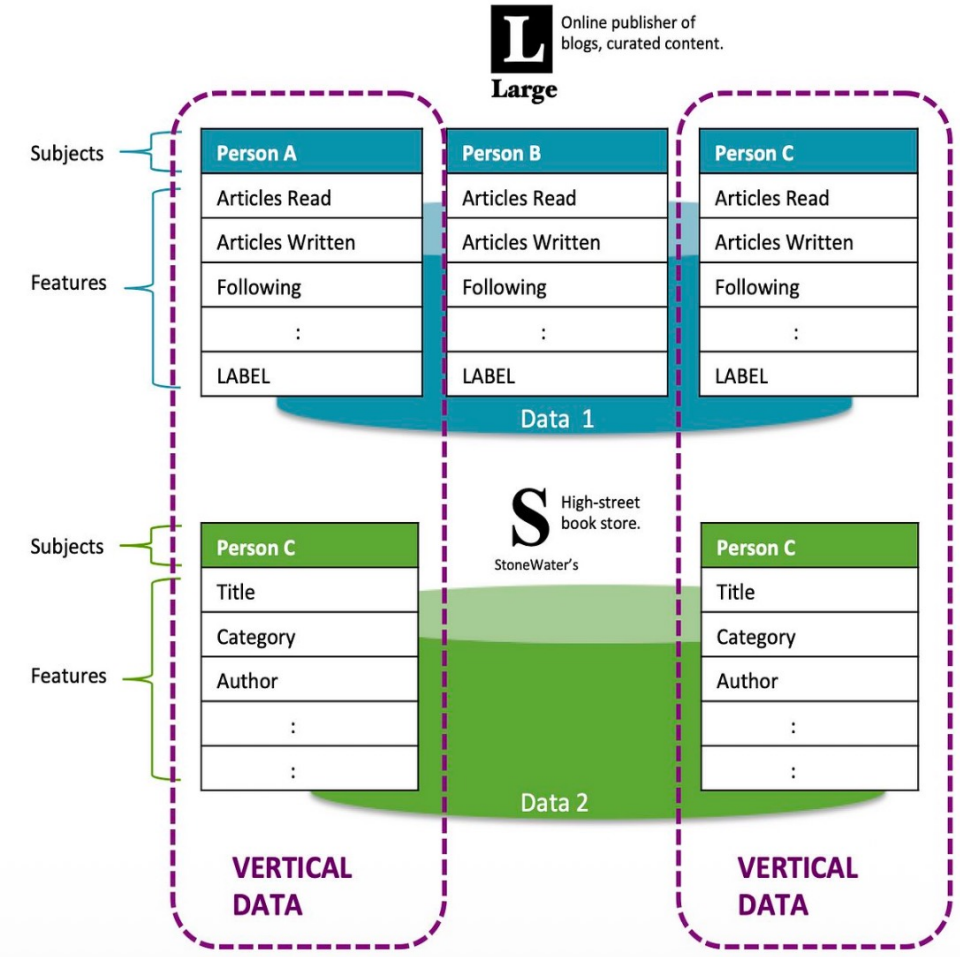
# Horizontal federated learning

- Shared features, different users: Clients have the same set of features.
- Focus: Leveraging the diversity of users with the same data structure to enhance model accuracy and generalization.
- Example: Multiple banks training a fraud detection model using transaction data (shared features) from different customers (different users).



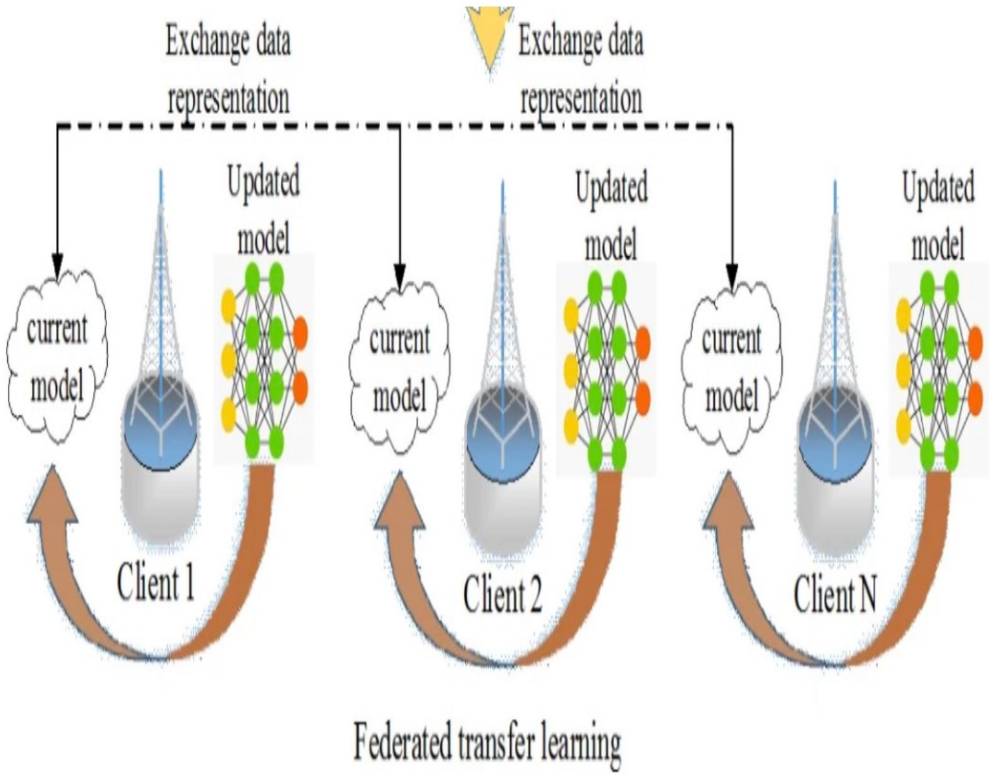
# Vertical federated learning

- Different features, overlapping users: Clients have different feature sets but some features might overlap.
- Focus: Combining data from participants with complementary information while protecting sensitive features.
- Example: Hospitals and insurance companies collaborating on healthcare predictions using medical records (Hospital data) and policy data (Insurance data) with overlapping features like patient IDs.



# Federated transfer learning

- Leveraging pre-trained knowledge: Uses a pre-trained model to guide learning on a new task or data with different characteristics.
- Focus: Accelerating learning on new tasks or data with limited resources, especially when privacy concerns restrict model sharing.
- Example: Using a sentiment analysis model trained on public product reviews to personalize recommendations within a specific e-commerce domain.



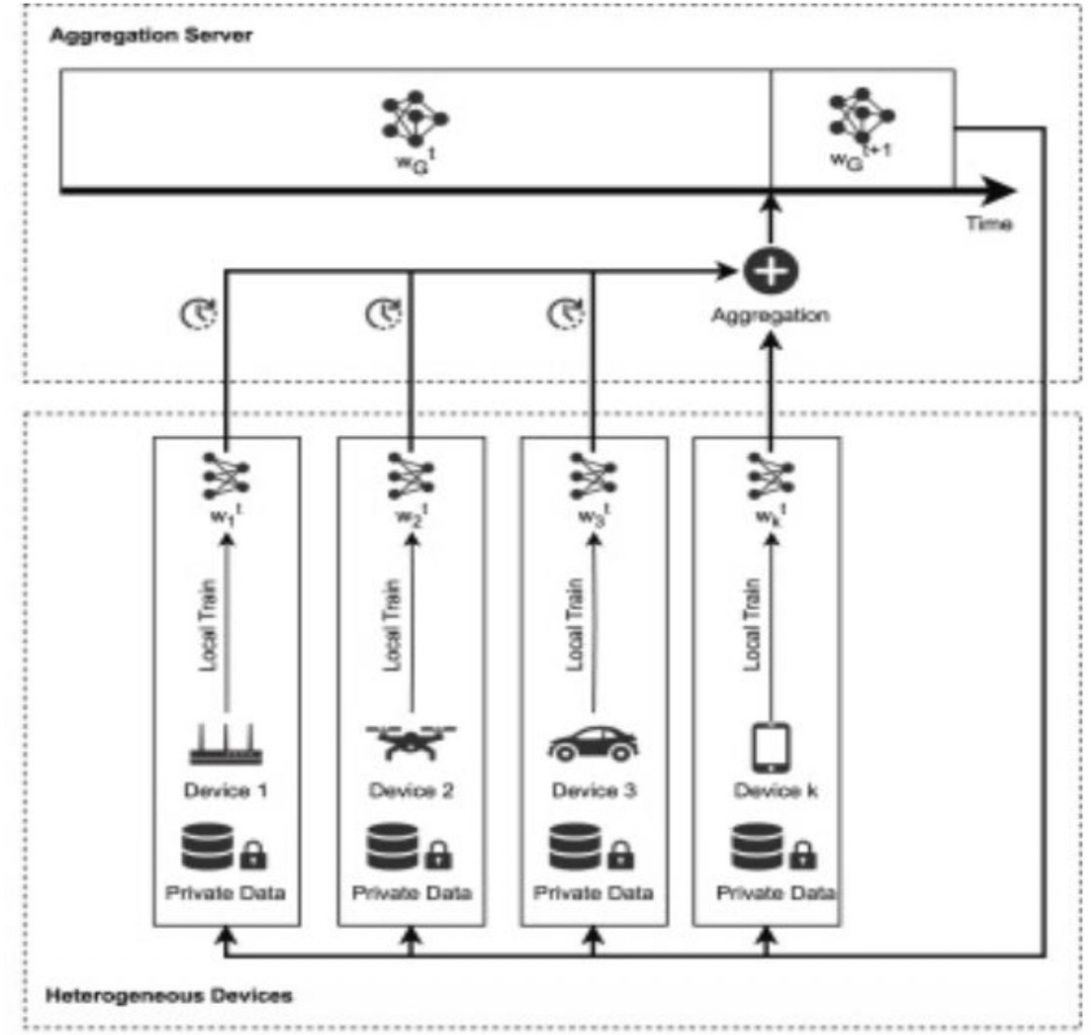
Federated transfer learning

# What is Key Differences?

| Features | Horizontal FL | Vertical FL | Transfer FL |
|---|---|---|---|
| Feature overlap | High | Low/Partial | No |
| User overlap | Low | High | Varies |
| Focus | Data diversity, Accuracy | Shared information, privacy | Knowledge transfer |

# Synchronous federated learning

- The Server updates the shared central model after “all the devices send their model updates”.
- Eg: Federated Averaging.

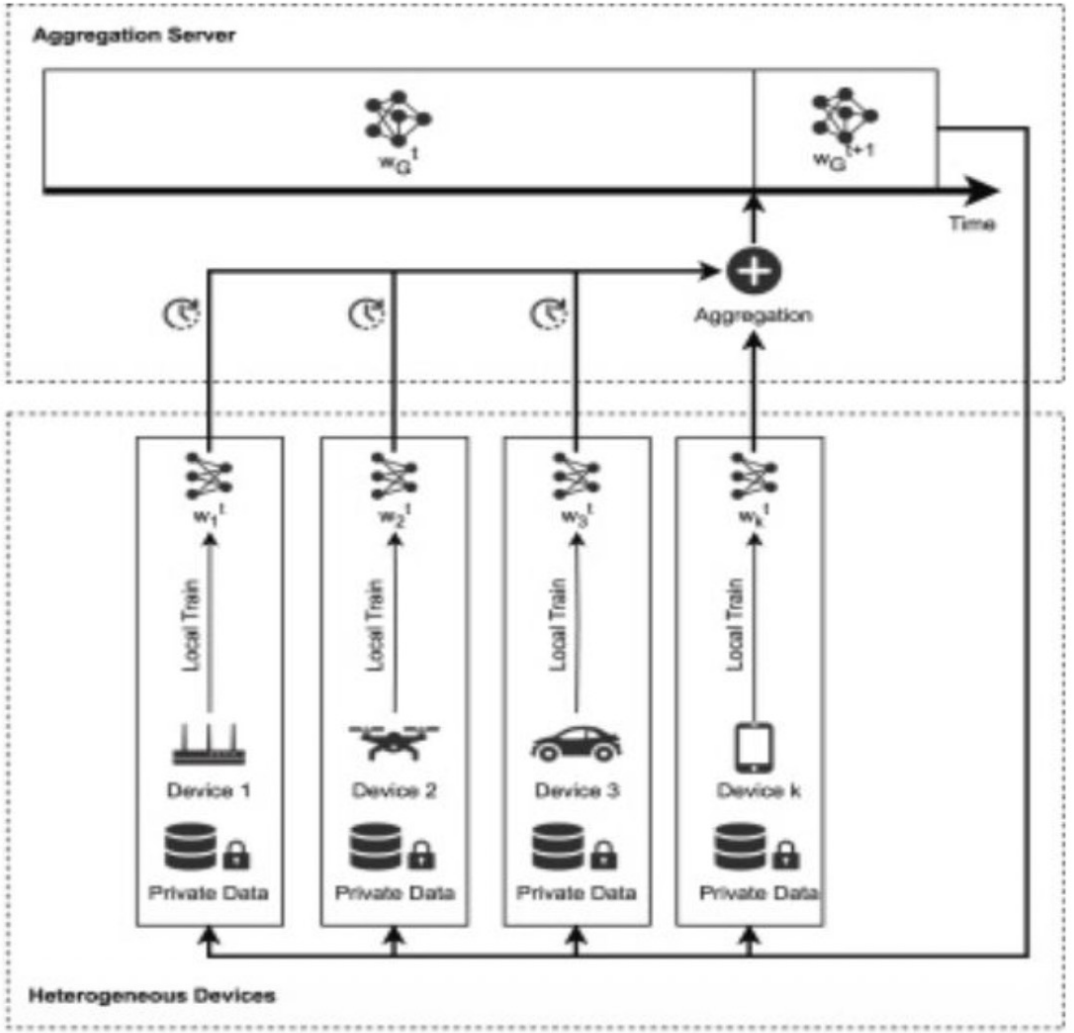



Synchronous Federated Learning

# Synchronous federated learning

This approach offers several advantages:

- Faster convergence: Synchronization leads to quicker convergence towards a more accurate global model.
- Better accuracy: The coordinated updates can result in higher model accuracy compared to asynchronous methods.
- Reduced staleness: Updates are always fresh, mitigating the issue of outdated gradients.



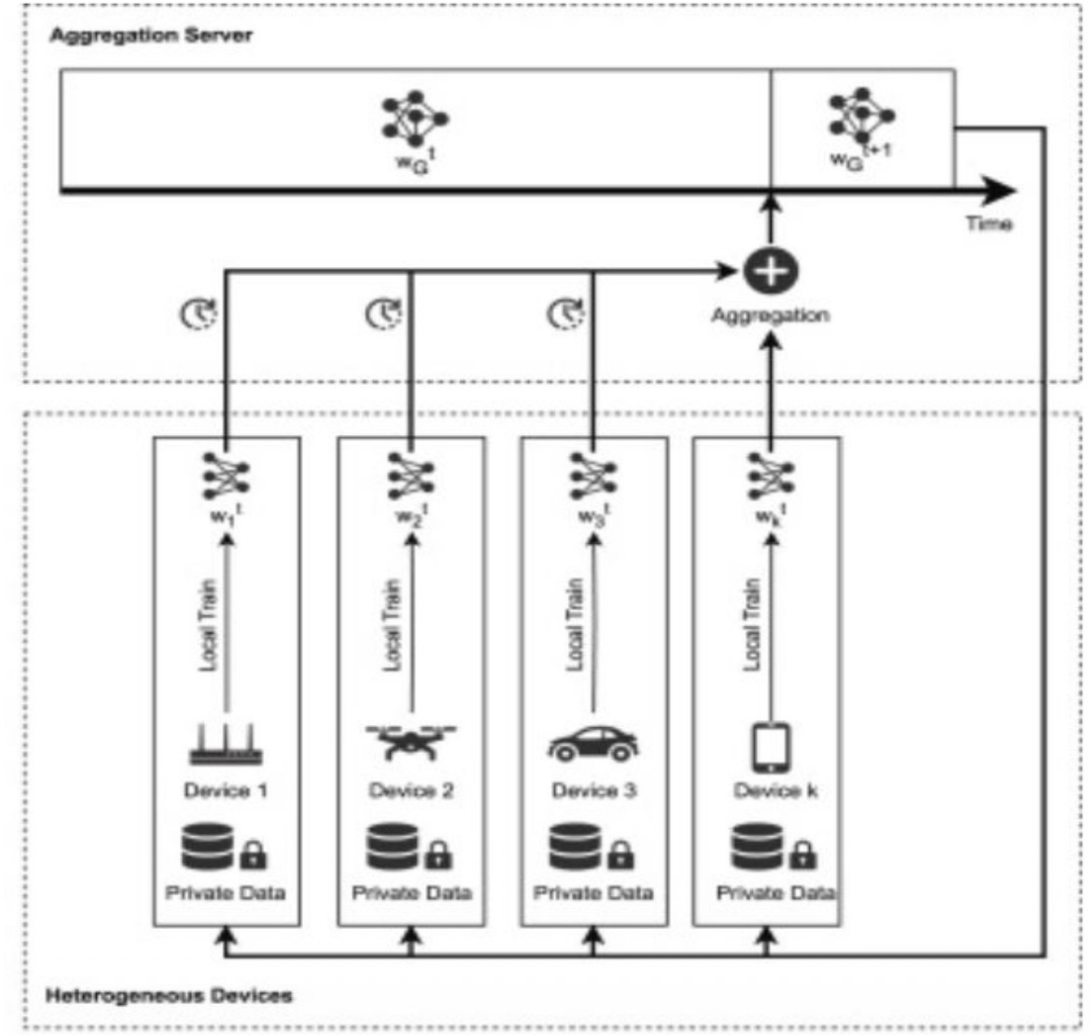

Synchronous Federated Learning

# Synchronous federated learning

However, synchronous federated learning

also faces some challenges:

- Increased communication overhead: All devices need to communicate with the server at every step, leading to higher bandwidth requirements.
- Higher synchronization latency: Waiting for the slowest device can introduce delays in the training process.
- Potential bottlenecks: Stragg

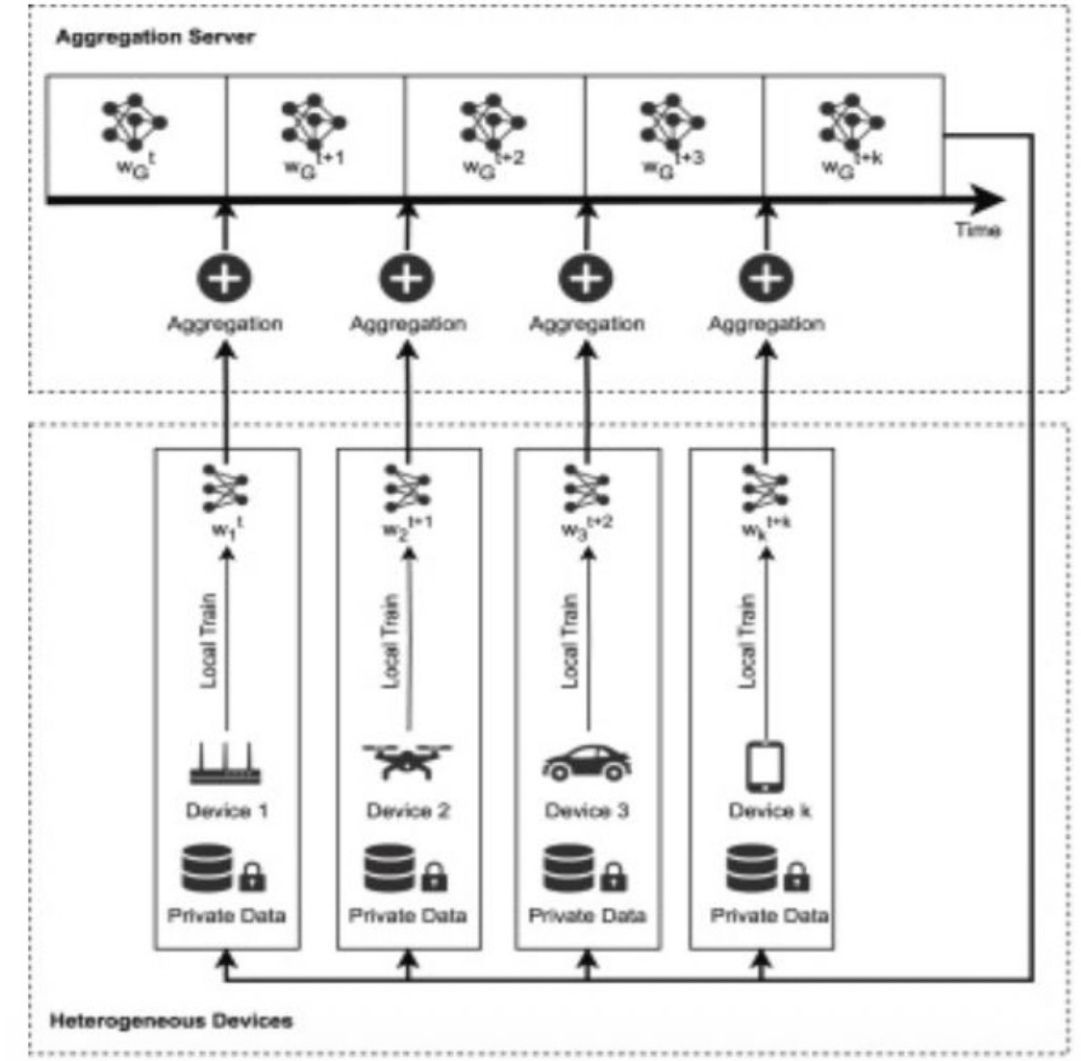



Synchronous Federated Learning

# Asynchronous federated learning

The Server updates the shared central model “as the new updates keep coming in”.

Eg: SMPC Aggregation, Secure Aggregation with Trusted Execution Environment(TEE).

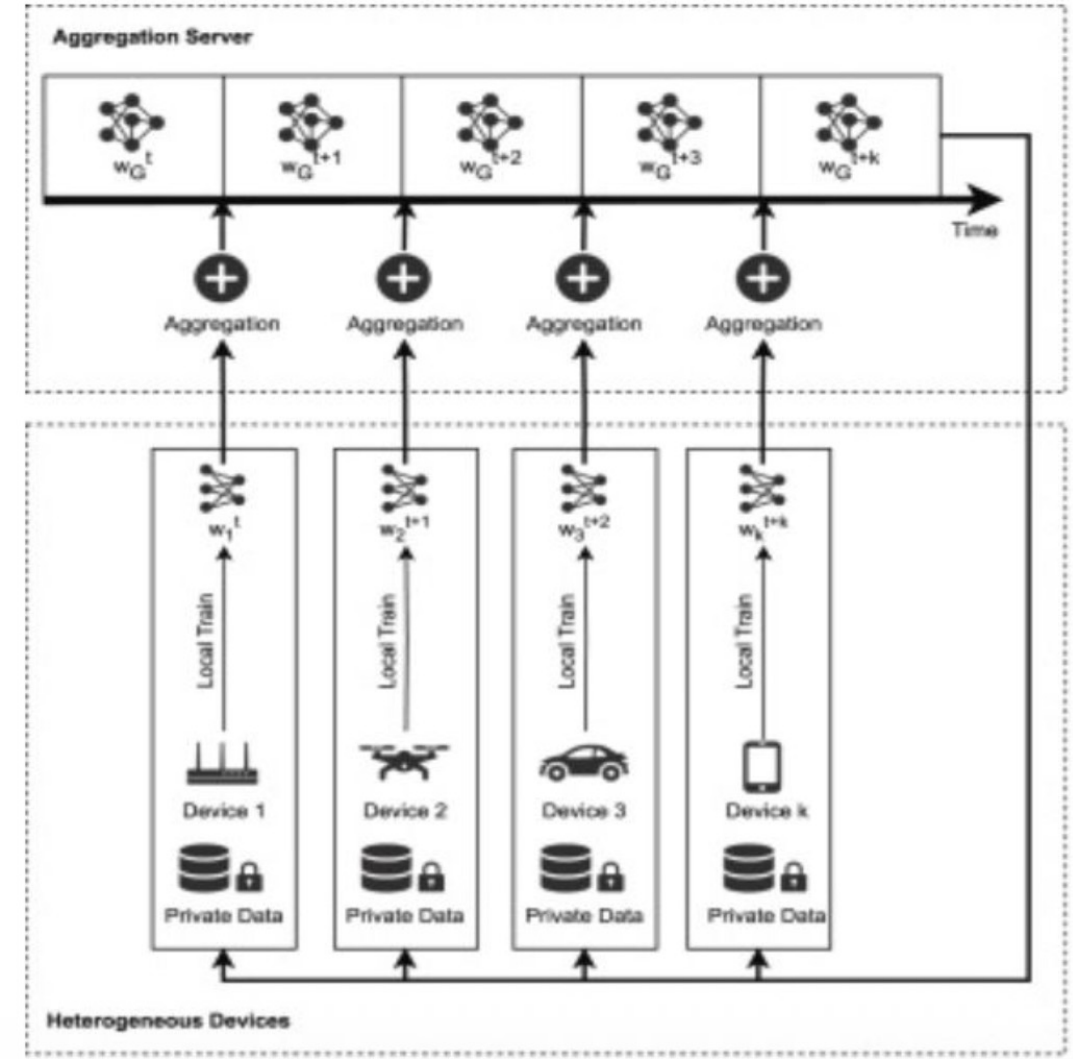



Asynchronous Federated Learning

# Asynchronous federated learning

This approach offers several advantages:

- Relaxed communication requirements: Devices can update the model whenever convenient, reducing communication overhead.
- Improved scalability: Asynchronous learning can handle a large number of devices more efficiently.
- Fault tolerance: The system is more resilient to device failures or intermittent connections.

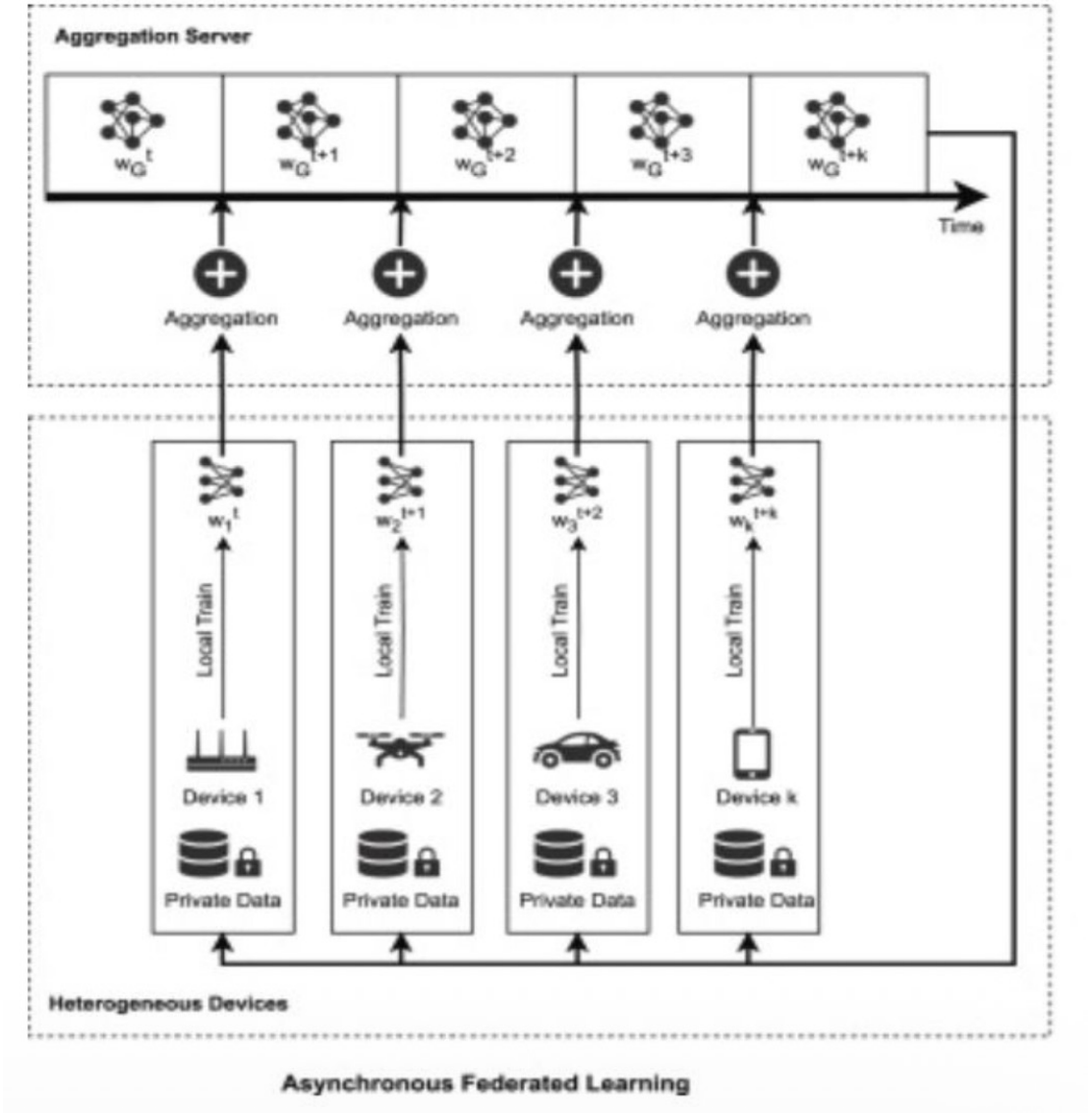



Asynchronous Federated Learning

# Asynchronous federated learning

However, asynchronous federated learning also faces some challenges:

- Stale gradients: Updates from devices may become outdated before reaching the server, impacting accuracy.
- Slower convergence: The lack of synchronization can slow down the overall training process.
- Potential for divergence: Individual models on devices may diverge significantly from the global model.




Asynchronous Federated Learning

# Synchronous FL and Asynchronous FL

| Aspect | Synchronous Federated Learning | Asynchronous Federated Learning |
|---|---|---|
| **Aggregation Timing** | Server waits for all selected clients to finish training before updating the global model | Server updates the global model immediately whenever a client sends its update |
| **Training Process** | Clients train in parallel in rounds | Clients train and send updates independently |
| **Server Update** | One global update per round | Continuous updates whenever updates arrive |
| **Speed** | Slower due to waiting for slow clients | Faster because server does not wait |
| **Straggler Problem** | Affected by slow or disconnected clients | Avoids the straggler problem |
| **Model Consistency** | More stable and consistent global model | Updates may use stale (old) models |
| **Communication Efficiency** | Less frequent updates | More frequent updates |

# What Is Aggregation in FL?

- Aggregation methods vary, each with unique advantages and challenges.
  - Beyond model updates, aggregate statistical indicators (loss, accuracy).
  - Hierarchical aggregation for large-scale FL systems.
- Aggregation algorithms are crucial for FL success.
  - Determine model training effectiveness.
  - Impact practical usability of the global model.

# Different Approaches of Aggregation

- Average Aggregation
- Clipped Average Aggregation
- Secure Aggregation
- Differential Privacy Average Aggregation
- Momentum Aggregation
- Weighted Aggregation
- Bayesian Aggregation
- Adversarial Aggregation
- Quantization
- Hierarchical Aggregation
- Personalized Aggregation
- Ensemble-based Aggregation

# Different Approaches of Aggregation

- Average Aggregation
- Clipped Average Aggregation
- Secure Aggregation
- Differential Privacy Average Aggregation
- Momentum Aggregation
- Weighted Aggregation
- Bayesian Aggregation
- Adversarial Aggregation
- Quantization
- Hierarchical Aggregation
- Personalized Aggregation
- Ensemble-based Aggregation

# Averaging Aggregation

- This is the initial approach and the most commonly known. In this approach, the server summarizes the received messages, whether they are model updates, parameters, or gradients, by determining the average value of the received updates.
- Since the set of participating clients is denoted by “N”and their updates are denoted by “$wi$”, the aggregate update “w” is calculated as follows:

$$w = (1/N) * \sum_{i=1}^{N} w_i$$

# Averaging Aggregation - Example

- Suppose 3 clients train locally and send their updated parameter (single weight) back to the server:
  - Client 1: n1 =50 samples, w1 =2.0
  - Client 2: n2 =30 samples, w 2 =1.0
  - Client 3: n 3 =20 samples, w 3 =3.0
- Three clients train locally and send updated vectors:
  - Client 1: n 1 =50 samples w 1 =[2.0,1.0,0.5]
  - Client 2: n 2 =30 samples w 2 =[1.0,2.0,1.5]
  - Client 3: n 3 =20 samples w 3 =[3.0,0.5,2.0]

# Clipped Averaging Aggregation

- This method is similar to average aggregation, where the average of received messages is calculated, but with an additional step of clipping the model updates to a predefined range before averaging.
- This approach helps reduce the impact of outliers and malicious clients that may transmit large and malicious updates.
- "$clip(x,c)$" is a function, which clips the values of "x" to a range of "$[-c,c]$", and "c" is the clipping threshold,

$$w = (1/N) * \sum_{i=1}^{N} clip\,(w_i, c)$$

# Class Task (Cont.)

- A national healthcare AI initiative aims to develop an edge detection model for detecting lung abnormalities from chest X-ray images. Due to patient privacy regulations, hospitals cannot share raw medical images. Instead, they adopt Horizontal Federated Learning (HFL) where each hospital trains the same model locally on its own dataset and only shares model parameters with the central server.
- In this system, the model consists of a single 3×3 convolution kernel used to extract edges from medical images. The central server initializes the model and sends it to participating hospitals. Each hospital trains the kernel using its own local X-ray dataset and then sends the updated kernel back to the server. The server aggregates the kernels using Federated Averaging (FedAvg) & Clipped Averaging to update the global model.

# Class Task

| Hospital | Dataset Description |
|---|---|
| Hospital A | Urban hospital with high patient volume |
| Hospital B | Specialized respiratory clinic |
| Hospital C | Rural hospital with diverse cases |

**Local Models Returned by Hospitals**

Hospital A kernel $W_1$

$$W_1 = \begin{bmatrix} 0.20 & 0.10 & 0.20 \\ 0.00 & 0.60 & 0.00 \\ -0.20 & -0.10 & -0.20 \end{bmatrix}$$

Hospital B kernel $W_2$

$$W_2 = \begin{bmatrix} 0.10 & 0.20 & 0.10 \\ 0.10 & 0.50 & 0.10 \\ -0.10 & -0.20 & -0.10 \end{bmatrix}$$

Hospital C kernel $W_3$

$$W_3 = \begin{bmatrix} 0.30 & 0.20 & 0.30 \\ -0.10 & 0.40 & -0.10 \\ -0.30 & -0.20 & -0.30 \end{bmatrix}$$

# Differential Privacy Average Aggregation

- This approach adds a layer of differential privacy to the aggregation process to ensure confidentiality of client data. Each client adds random noise to its model update before sending it to the server, and the server compiles the final model by aggregating the updates with the random noise.
  - "$ni$" is a random noise vector, drawn from a Laplace distribution
  - "b" is a privacy budget parameter,.

$$w = \frac{1}{N} \sum_{i=1}^{N} (w_i + b \cdot n_i)$$

# Differential Privacy

- Differential Privacy (DP) is used to protect the privacy of individual clients participating in federated learning. In this approach, after collecting the local model updates from all clients, the server first performs a standard weighted averaging to obtain the global model. To ensure privacy, a small amount of random noise (usually Gaussian noise) is then added to the aggregated model parameters before updating the global model.
- This noise prevents attackers from inferring whether a specific client's data contributed to the model update, thereby protecting sensitive information while still allowing the model to learn useful patterns. The privacy level is controlled by the privacy budget (ε), where smaller values provide stronger privacy but may slightly reduce model accuracy.

# DP Avg

| Hospital | Dataset Description | Number of Images |
|---|---|---|
| Hospital A | Urban hospital with high patient volume | 600 X-ray images |
| Hospital B | Specialized respiratory clinic | 300 X-ray images |
| Hospital C | Rural hospital with diverse cases | 100 X-ray images |

**Local Models Returned by Hospitals**

Hospital A kernel $W_1$

$$W_1 = \begin{bmatrix} 0.20 & 0.10 & 0.20 \\ 0.00 & 0.60 & 0.00 \\ -0.20 & -0.10 & -0.20 \end{bmatrix}$$

Hospital B kernel $W_2$

$$W_2 = \begin{bmatrix} 0.10 & 0.20 & 0.10 \\ 0.10 & 0.50 & 0.10 \\ -0.10 & -0.20 & -0.10 \end{bmatrix}$$

Hospital C kernel $W_3$

$$W_3 = \begin{bmatrix} 0.30 & 0.20 & 0.30 \\ -0.10 & 0.40 & -0.10 \\ -0.30 & -0.20 & -0.30 \end{bmatrix}$$

Assume the server samples the following Gaussian noise matrix:

$$Z = \begin{bmatrix} 0.01 & -0.02 & 0.00 \\ -0.01 & 0.03 & -0.02 \\ 0.02 & -0.01 & 0.01 \end{bmatrix}$$